Indexed PHOAS interpretation and folds for PTS-rk

October 22, 2026

Abstract

The full annotated PTS-rk admits normal type and term representations whose identity interpretation is originally typed and reduces to the literal, possibly reducible source. The interpreter is indexed by a rank budget and a finite numerical equality plan. Its common operation is a fold: application of a represented program to an originally typed filling of a dependent telescope. Types and terms use the same algebra. Neither raw syntax tags nor numerical encodings are needed for evaluation.

This shared API does not yet give one constructor signature per rank fragment. We distinguish strong recovery from intrinsic syntax fidelity, and a uniform execution head from a uniform algebra. Checked alternative interpretations include a constructor observer on faithful simply typed PHOAS and a continuation-passing fold on binder-free trees. The latter and a Boolean constructor test consume the same rank-correct code. Full dependent alternative algebras, intrinsic fidelity for arbitrary dependent payloads, and internal reification of accepted proof codes remain precise open obligations.

1 Introduction

Brown and Palsberg's *Breaking Through the Normalization Barrier: A Self-Interpreter for* F_ω[1] separates external quotation from internal interpretation. Normal syntax representations can describe reducible programs; applying an interpreter to such a representation recovers the original program. This is stronger than returning a code for its normal form.

For PTS-rk, the basic interface is an algebra for a finite dependent telescope Σ. An algebra is a tuple σ of ordinary PTS terms satisfying the declarations of Σ. A closed representation has the form $\lambda\Sigma.q$, and its fold is ordinary application:

$$\mathsf{fold}_\sigma(Q) = Q\,\sigma, \qquad \mathsf{fold}_\sigma(\lambda\Sigma.q) \longrightarrow_\beta^* q[\sigma].$$

The same operation interprets represented types and represented terms. It traverses an already constructed object-language representation by beta reduction, not by a new Rocq traversal of the source. Algebra supplies typing, substitution and conversion laws; GenericFold packages specialization as a normal head receiving the algebra fields as ordinary arguments. UnifiedAlgebra provides full dependent quotation and identity interpretation; PHOASFold and CombinatoryAlgebra instantiate this API with alternative interpretations. A common operation does not make their signatures identical.

The normalization bounds explain why a non-uniform construction is plausible. For a fixed redex-rank bound d, the checked normalizer has

$$\mathrm{cost}(\mathsf{normalize}_d(t)) \le \mathrm{tower}_{2d+1}(N + 5d + 2), \quad \mathrm{tower}_0(x) = x, \quad \mathrm{tower}_{h+1}(x) = 2^{\mathrm{tower}_h(x)}.$$

[1] https://doi.org/10.1145/2837614.2837623

Here N counts nodes; every closed typed term has the computable sufficient bound $1 + \mathsf{maxRankLiteral}(t) + \mathsf{nodes}(t)$. Thus each fixed-bound normalizer is elementary. The manuscript's elementary representability argument would implement its numerical computation at a computable finite rank budget. The generic compiler needed for this route is not a Rocq theorem. Nor does numerical normalization alone produce an inhabitant of a represented dependent type. The interpreter below establishes typed recovery directly, without that compiler and without pre-normalizing the source term.

2 Dependent algebras and full interpretation

Terms and types share the original annotated syntax

$$t ::= x \mid \mathcal{U}_i \mid \Pi x : t.t \mid \lambda x : t.t \mid t\,t.$$

We use original typing and beta reduction, including reduction inside annotations. The characteristic rules are

$$\Gamma \vdash \mathcal{U}_i : \mathcal{U}_0, \qquad \frac{\Gamma \vdash A : \mathcal{U}_a \quad \Gamma, x : A \vdash B : \mathcal{U}_b}{\Gamma \vdash \Pi x : A.B : \mathcal{U}_{\max(a+1,b)}}.$$

There is no cumulativity, eta conversion, additional primitive or axiom. Literal source equality includes annotations.

2.1 One fold for types and terms

For $\Sigma = (x_1 : S_1, \ldots, x_m : S_m)$, a filling σ is required to satisfy each declaration after substitution of the preceding fields. This is the original typed-substitution judgment. The Rocq algebra record packages that evidence; it does not add an object-language record type or assume semantic soundness.

The dependent signature supplies code carriers C_s and decoding families El_s. Schematically, its two program types are

$$\mathsf{TyProg}_{\Sigma,s} = \Pi\Sigma.C_s,$$
$$\mathsf{TmProg}_{\Sigma,s}(P) = \Pi\Sigma.\mathsf{El}_s(P\,\Sigma).$$

Actual annotations use cached normal bodies, so that apparent applications of P do not introduce annotation redexes into the constructed programs. For any typed algebra σ, specialization gives

$$\frac{P : \mathsf{TyProg}_{\Sigma,s} \qquad Q : \mathsf{TmProg}_{\Sigma,s}(P)}{\mathsf{fold}_\sigma(Q) : \mathsf{El}_s[\sigma](\mathsf{fold}_\sigma(P))}.$$

Algebra proves the closure beta law, compatibility with substitution, extensionality of fillings and preservation of conversion. UnifiedAlgebra proves the above dependent specialization theorem for the generated full-source programs, not just for a separately defined host-language evaluator.

GenericFold packages specialization into one originally typed head for each signature, with no algebra filling built into it:

$$G_{\Sigma,s} = \lambda P : \mathsf{TyProg}_{\Sigma,s}.\lambda Q : \mathsf{TmProg}_{\Sigma,s}(P).\lambda\Sigma.Q\,\Sigma,$$
$$G_{\Sigma,s}\,P\,Q\,\sigma \longrightarrow^*_\beta \mathsf{fold}_\sigma(Q).$$

It is normal for the generated normal signature and neutral decoding family. If $\rho_\Sigma(u)$ is the telescope's closure rank at result rank u, its type-sort is $\max(\rho_\Sigma(0)+1, \rho_\Sigma(s)+1)$. The fields of σ are ordinary remaining arguments, not an external evaluator parameter. The syntactic composition theorem reduces nested invocations to nested folds; it asserts neither semantic fusion nor typing compatibility between unrelated signatures. One head per signature is not one constructor signature for an entire rank fragment.

Theorem 2.1 (Tag-free full dependent recovery)**.** *For every original closed judgment $\vdash t : T$, quotation supplies finite numerical indices k, p, s, with $s \le k+1$, and closed normal programs P, Q over a signature $\Sigma_{k,p}$. Its constructed normal identity algebra ι satisfies*

$$\begin{aligned} &\mathsf{fold}_\iota(P) \longrightarrow^*_\beta T,\\ \vdash\ &\mathsf{fold}_\iota(Q) : T,\\ &\mathsf{fold}_\iota(Q) \longrightarrow^*_\beta t. \end{aligned}$$

The corresponding normal, originally typed head $I_{k,p,s}$ takes the ordinary arguments P, Q and satisfies $I_{k,p,s}\,P\,Q \longrightarrow^*_\beta t$. No raw HOAS tag is used in these programs or their interpretation.

Quotation is executable on independent six-rule derivation data. Coverage of original propositional typing is a separate theorem: the quoter does not inspect an erased Rocq proposition. The indices describe rank and equality-field allocation, not the source syntax or a source term supplied secretly to the interpreter. For a well-formed open context, quote the literal dependent closure $\lambda\Gamma.t$ and reopen its recovered value at the variables of Γ. Original typing and literal recovery hold in that context.

2.2 Recovery and fidelity are different

For a common algebra, convertible represented programs have convertible recovered sources. If those sources are normal, this gives literal injection. It does not give literal injection for arbitrary reducible sources: distinct beta-equivalent terms can have a common reduct.

The simply typed PHOAS codes and the binder-free codes below already have intrinsic structural syntax-injectivity proofs. For the full dependent payload, literal injection across arbitrary repairs and plans is not yet proved. The compatibility interface FullDependentInterpreter therefore retains a separate faithful HOAS envelope when that stronger contract is needed. Its established cross-plan annotated-source injection has not been discarded or silently transferred to the tag-free interface.

Removing this envelope while keeping the same fidelity theorem requires a structural decoder for the actual payload, stable under binders, repairs and signature insertion. An external quotation tree is useful metadata, but is not such a decoder. No impossibility result for this stronger intrinsic dependent fidelity is claimed.

3 Why dependent quotation needs coherence fields

The signature distinguishes type codes from their values:

$$C_a : \mathcal{U}_0, \qquad \mathsf{El}_a : C_a \to \mathcal{U}_a.$$

A dependent-product constructor takes a code $A : C_a$ and a family $B : \mathsf{El}_a(A) \to C_b$, with result in $C_{\max(a+1,b)}$. A quoted binder is a genuine target binder. Universe codes, abstraction and application operations complete the basic signature.

To relate product decoding to actual dependent functions, and universe decoding to actual universes, the signature also contains equality laws. At suitable ranks these use Leibniz equality

$$\mathsf{Eq}_{A,d}(x, y) = \Pi P : (A \to \mathcal{U}_d).\, P\, x \to P\, y.$$

For $A : \mathcal{U}_a$, its sort is $\mathcal{U}_{\max(a+2,d+1)}$. Product coherence uses sufficiently high motives to transport in both directions. Under the identity interpretation, C_a becomes $\mathcal{U}_a$, El_a becomes the identity, and the constructors become the original PTS operations; the law witnesses reduce to reflexivity.

Before filling the signature, however, these equality fields are neutral. A transport followed by its inverse need not reduce to the original term. Consequently, the code family expected by a quoted dependent application need not be convertible to the family obtained by quoting the source substitution.

The construction resolves this with a finite telescope of contextual repairs. A repair is a field $\Pi\Delta.\mathsf{Eq}(u, v)$ whose original typing is checked, and whose filled endpoints are independently proved convertible. Normalizing a constructed equality witness supplies a normal identity filling; it does not normalize the represented source. The allocation plan records only numerical scope, node-count, literal and rank budgets, not endpoint terms or proof objects.

The executable quotation follows all six original rules:

- Sorts and variables use the universe and variable components.
- Products and lambdas quote their components under actual binders.
- Application uses two checked repairs and substitution on finite quotation views, aligning both the function family and the result.
- Conversion quotes the required formation data and introduces the corresponding checked alignment.

Application does not recursively quote an arbitrarily synthesized derivation. This matters because reconstructing formation derivations can otherwise create cycles around the rule $\mathcal{U}_i : \mathcal{U}_0$. The finite view/substitution lemmas and structural recursion on the independent input derivation close this gap. Closing the telescope then produces the normal type and term programs used above.

These are mathematical coverage and executability results, not a performance claim. The extracted sort path has been exercised; lambda and application probes reached a 30-second timeout. Pruning literal reflexive conversion steps removes avoidable fields, but does not establish derivation-independent quotation.

4 Alternative PHOAS folds and continuations

4.1 An actual nonidentity interpretation

For the simply typed fragment over a ground type $O : \mathcal{U}_0$, the existing normal PHOAS code abstracts over rank-preserving families $F_a : \mathcal{U}_a \to \mathcal{U}_a$ and typed abstraction/application handlers. PHOASFold applies an already produced code to any such typed filling, and proves its exact agreement with Algebra's fold.

A concrete filling observes the outer constructor. Put

$$D_0 = \mathcal{U}_0, \qquad D_{a+1} = D_a \to \mathcal{U}_0, \qquad F_a(A) = D_a.$$

For each a, choose two closed normal observations $o_a^0, o_a^1 : D_a$: set $o_0^i = \mathcal{U}_i$ and $o_{a+1}^i = \lambda x : D_a.\mathcal{U}_i$ for $i = 0, 1$. They are not beta-convertible. The abstraction handler returns $o^0_{\mathrm{ord}(A\to B)}$; the application handler returns $o^1_{\mathrm{ord}(B)}$. These are exact-rank, originally typed handlers. Folding the normal quoted program returns the corresponding observation by beta reduction.

This is not the identity interpretation in disguise, a new source traversal, or a numeral encoding. Nor are these two observations claimed to be an eliminable Church Boolean: their nonconvertibility suffices for the theorem.

There is a checked regression at the original source level:

$$(\lambda f : O \to O.f)(\lambda x : O.x) \longrightarrow_\beta^* \lambda x : O.x.$$

The two faithful normal quotations give distinct observer outputs. Thus this observer does not preserve all source beta equations. PHOASConversion proves the related general obstruction: a shared map cannot simultaneously be normal-valued, injective on literal syntax and preserve all source conversion. This excludes that conjunction, not PHOAS itself, separate syntax and semantic indices, or transports.

4.2 What the continuation construction proves

For $A : \mathcal{U}_a$ and an answer type $R : \mathcal{U}_b$, define

$$C_R(A) = (A \to R) \to R, \qquad h(a, b) = \max(a + 2, b + 1).$$

PHOASCPS proves $C_R(A) : \mathcal{U}_{h(a,b)}$ in the original calculus. Its call-by-name local handlers, writing $D = C_R(A) \to C_R(B)$, are

$$\begin{aligned} \mathsf{ret}(v) &= \lambda k.k\, v, \\ \mathsf{abs}_C(f) &= \mathsf{ret}(f), \\ \mathsf{app}_C(f, x) &= \lambda k.f\, (\lambda g : D.g\, x\, k). \end{aligned}$$

Here $f : D$ in the abstraction clause and $f : C_R(D), x : C_R(A)$ in the application clause. All annotations and rank inequalities are checked. The administrative beta theorem gives

$$\mathsf{app}_C(\mathsf{abs}_C(f), x)\, k \longrightarrow_\beta^* f\, x\, k,$$

and, when $f = \lambda x : C_R(A).u$, the right side reduces to $u[x := x_0]\, k$ for the supplied argument x_0. No eta conversion is used. These are actual local PTS terms and typing theorems, not a complete PHOAS CPS translation.

The structural call-by-name type translation is also originally typed:

$$V(O) = O, \qquad V(A \to B) = C_R(V(A)) \to C_R(V(B)).$$

Its computation translation is $C_R(V(A))$. Already for $O, R : \mathcal{U}_0$, a right-associated chain A_n of $n \geq 1$ ground arrows has source order 1 but translated computation rank $2n + 3$. The exact sort theorem and sort uniqueness therefore rule out one target rank budget for this particular translation of the whole source-order-one fragment. They also rule out using C_R directly as the existing same-rank family F_a. This does not rule out every rank-indexed CPS encoding.

4.3 The full dependent alternative-algebra obligation

A full dependent algebra must fill not only constructors but their coherence and repair fields. Replacing El or the term carrier changes those obligations. An arbitrary constructor observer cannot simply reuse identity witnesses; nor does nonconvertibility of two endpoints by itself prove that an equality type is uninhabited.

Dependence makes the issue substantive: evaluating a term whose type is $B(u)$ must track how the transformed argument determines its result type. A promising next construction is a semantic-indexed or glued algebra, retaining a direct semantic value alongside an observation or continuation computation. Its binder, substitution and conversion laws must be established before it can fill the existing dependent interface. Such an algebra, and a full dependent CPS correctness theorem, remain open. Wrapping the identity evaluator in a continuation is not an alternative constructor algebra and would not discharge this obligation.

5 Binder-free folds and the combinatory connection

The variable-free representation separates applications from a finite bank of atoms. For an m-entry bank, CombinatoryFold constructs

$$\mathsf{TreeCode}_{r,m} = \Pi X : \mathcal{U}_r.\, (X \to X \to X) \to \underbrace{X \to \cdots \to X}_{m \text{ leaves}} \to X.$$

Its sort is $\mathcal{U}_{r+2}$. A quoted tree is a closed normal original PTS term, and equality of such codes up to beta conversion implies equality of their trees. Its own syntax carries the information: no auxiliary HOAS tag or conversion to numerals is involved.

For $X : \mathcal{U}_r$, operation $\circ : X \to X \to X$ and leaf values $a_i : X$, the internal operation is simply

$$\mathsf{run}(q, X, \circ, \vec{a}) = q\, X\, \circ\, a_1 \cdots a_m.$$

Applied to a quoted tree, it reduces to the corresponding expression $\mathcal{I}_{\vec{a},\circ}(t)$. The external tree fold specifies this result; it is not the internal implementation. CombinatoryAlgebra proves that run is exactly the shared typed-telescope fold.

5.1 CPS and a constructor test on the same code

The homogeneous tree algebra supports genuine call-by-value CPS. For $X : \mathcal{U}_a$, $R : \mathcal{U}_b$, set $c = h(a, b)$ and use carrier $C_R(X) : \mathcal{U}_c$, leaf values $\mathsf{ret}(a_i)$, and operation

$$f \circ_C x = \lambda k.f\,(\lambda v.x\,(\lambda w.k\,(v \circ w))).$$

This is an originally typed filling for a code at rank c. Specializing the already constructed code yields the checked theorem

$$\mathsf{cpsFold}(\ulcorner t \urcorner) \longrightarrow^*_\beta \lambda k.k\,\mathcal{I}_{\vec{a},\circ}(t).$$

Applying this result to k reduces to $k\,\mathcal{I}_{\vec{a},\circ}(t)$. The theorem is proved by the constructor-fold equations, not by defining CPS to be a wrapper around a precomputed evaluation. It concerns the interpreted tree expression, not a claim that this expression is normalized or that arbitrary dependent source CPS is solved.

A second filling tests whether the tree is a leaf or an application, using actual Church selectors. The two interpretations consume the same literal code $\mathsf{TreeCode}_{c,m}$: use the Boolean carrier

$$\mathsf{Bool}_{c-1} = \Pi Z : \mathcal{U}_{c-1}.Z \to Z \to Z : \mathcal{U}_c.$$

Since $c = \max(a + 2, b + 1) \geq 2$, this is well-defined with exactly the required rank. Neither cumulativity nor re-quotation is used. A first-leaf fold supplies a further simple nonidentity interpretation. These are homogeneous applicative-tree folds, not yet dependent source-type-indexed interpretations.

5.2 What the combinatory bridge supplies

A combinatory source is factored into a finite dictionary of universe and scheme atoms and a tree of dictionary references. The expansion theorem recovers the literal original combinatory code. For a fixed dictionary, conversion-injectivity of tree codes therefore implies equality of the expanded syntax; different dictionaries are not silently identified.

The bridge applies to the independent Hilbert deduction system, and specializes to the concrete sound 65-shape catalogue in CombinatoryLogic-rk. From its derivations it obtains an expansion with original PTS typing and recovery, together with a normal typed tree code. Atoms are actual universe and catalogue instances, not arbitrary source terms inserted as opaque atoms.

The dictionary is finite per source, not universal. Soundness of this catalogue does not establish completeness for all original PTS derivations, general dependent bracket abstraction, or a homogeneous algebra evaluating arbitrary dependently typed atoms. Those remain separate obligations. Nevertheless, this bridge gives a simpler syntax/fold interface for independent proof certificates and a useful point of contact with the internal-consistency project.

6 Uniformity over rank-bounded fragments

6.1 The obstruction and the positive bounds

The type rank of $t : T$ is the unique a with $T : \mathcal{U}_a$. Application cannot increase it: $\mathrm{rk}(f\,u) \leq \mathrm{rk}(f)$. Thus no one closed typed application head can recover every closed typed term. Indeed, with D_a as above, normal inhabitants $d_a : D_a$ exist for every a, whereas every spine headed by a fixed E has type rank at most $\mathrm{rk}(E)$. Ordinary rank numerals among its arguments do not change this obstruction.

For a fixed budget k, three different uniformity questions remain:

1. Can all sources share a bound on interpreter rank?
2. Can they share the same execution head?
3. Can they share a constructor signature admitting other algebras?

The answer is positive to the first two for the checked quotation-budget fragment. The third is open.

The fragment bounds the pruned independent derivation's quotation budget, including formation ranks and universe literals, not syntax size. Every original source belongs to some such fragment; open terms are handled after dependent closure. This is not automatically the fragment defined solely by the normalizer's redex-rank parameter.

The capped construction bounds local alignment fields by $k+2$ and the base declarations by the $k+3$ cap. Dependent telescope rank uses a maximum rather than a sum, so its length need not increase the bound. The faithful compatibility interpreter has type-sort bounded by

$$R(k, r) = \max(k + 7, r + 6),$$

independently of plan length, where r is the optional syntax-tag carrier rank. This theorem does not confer the envelope's stronger fidelity on the tag-free payload.

The current allocation plan can have unbounded length at fixed small ranks, even for a fixed final judgment with conversion detours. An exhaustive finite list of concrete equality fields likewise cannot cover all contexts at a fixed rank. These checked obstructions concern the current plan and literal catalogues, not finite generic coherence schemes or every possible rank-indexed PHOAS algebra.

6.2 A common execution head

FixedInterface uses a typed closure encoding

$$\begin{aligned} H_a(T) &= \Pi A : \mathcal{U}_a.\, A \to (A \to T) \to T, \\ \mathsf{Code}_a(T) &= H_a(T) \to T. \end{aligned}$$

A code $\lambda h.h\,A\,q\,f$ hides a program $q : A$ and its runner $f : A \to T$. The normal decoder applies it to $\lambda A\,x\,f.f\,x$. Its polymorphic head $E_{a,s} : \Pi T : \mathcal{U}_s.\mathsf{Code}_a(T) \to T$ has type-sort $\max(a+4, s+3)$. The checked embedding and decoder give $T \leftrightarrow \mathsf{Code}_a(T)$ and a beta round trip.

Exact-rank padding avoids cumulativity: $\mathsf{Pad}_k(A) = D_k \to A : \mathcal{U}_{k+1}$ when $A : \mathcal{U}_a$ and $a \leq k+1$. Embed a value by a dummy abstraction and force it at d_k. FragmentClosureInterpreter fixes the head $E_{k+6,k+1}$ before the source, with type-sort $k + 10$, and obtains

$$E_{k+6,k+1}\,(\mathsf{Pad}_k(\mathsf{nf}(T)))\,q\,d_k : T \quad \longrightarrow_\beta^* \quad t.$$

Only the type T is externally normalized, not the source term t. The final force is typed using the supplied padded function shape; it is not an eliminator for arbitrary types in $\mathcal{U}_{k+1}$. Its ordinary type argument is not the literal-recovering type program P; that separate type-recovery theorem remains available.

This establishes uniform execution for the whole fragment. The varying runner and its plan are carried by the code, and the compatibility package retains annotated-source injection. A common head does not make that hidden runner a common constructor algebra, nor does it construct an internal proof-code reifier.

7 Numerals and internal consistency

None of the dependent, simply typed or combinatory folds above requires converting source syntax into numerals. Finite telescopes and finite dictionary choices suffice. Rank bounds and allocation plans are still external resources; avoiding numeric payloads does not internalize them. FiniteLiteralBank supplies structural selectors for a fixed universe bank. NumericLiteralBank is an optional, saturating Church-numeral adapter, not part of the evaluation interface.

For an internal consistency argument, the crucial additional operation has a different input. Fix an independently specified proof-code domain D, an accepted-certificate predicate $\mathsf{Accepted}_k$, and $\mathsf{Bottom} = \Pi X : \mathcal{U}_0.X$. The desired object-language terms are

$$e_k : \mathsf{Code}_k(\mathsf{Bottom}) \to \mathsf{Bottom}, \qquad q_k : \Pi p : D.\mathsf{Accepted}_k(p) \to \mathsf{Code}_k(\mathsf{Bottom}).$$

The composite would turn an accepted contradiction certificate into Bottom. The predicate must describe independent proof rules, trace witnesses and resource bounds, and the theorem must cover arbitrary accepted inputs, not just externally generated examples. The output must have literal type Bottom, or an internally justified equal type.

An external Rocq quotation function is not q_k. Furthermore, $\mathsf{Code}(T)$ is inhabited exactly when T is: postulating reification into $\mathsf{Code}(\mathsf{Bottom})$ would assume the missing soundness content. The common head supplies execution once a code exists, not the accepted-proof theorem. A Bottom-specific tag-free interface may be enough; general fidelity and CPS are not prerequisites for this use.

There is also a checked literal-alphabet limit. Beta reduction cannot create a universe literal absent from the original syntax. If fixed reifier and evaluator heads consume inputs with a uniformly bounded literal alphabet, every recovered normal term has literal maximum bounded by the maximum of these fixed bounds. Consequently no such numeral-only interface can unquote every $\mathcal{U}_i$, although each $\mathcal{U}_i$ has type rank zero and redex bound zero. This does not obstruct fixed fragments, a finite literal bank, or Bottom-specific soundness. The optional numeral adapter selects $\mathcal{U}_{\min(i,k)}$ on the displayed standard numerals; it is neither unbounded literal generation nor a general certificate validator.

The summary in theories/self_consistency separates two proof routes: an interpretation of $H^{\omega}_{<\omega}$ followed by explicitly unverified claims from Pakhomov's transfer theorem, and a direct resource-restricted internal interpretation of proof certificates. Shared folds and the combinatory dictionary bridge can simplify the representation layer in either route. They do not supply internal induction, admissible arithmetic domains, trace soundness, or internal budget selection. An external family of results indexed by k is not yet one internal proof quantifying over represented budgets.

8 Comparison with Brown–Palsberg and the remaining target

Brown–Palsberg's fold applies a normal PHOAS representation to a type constructor and four handlers: term abstraction/application and type abstraction/application. Their substitution theorem explains its computation on prequotations. The unquoter is one polymorphic term, taking an ordinary represented type and term as arguments. Our fold uses the same specialization mechanism, and full dependent recovery likewise consumes both arguments, but finite rank and coherence signatures replace their one polymorphic signature.

Their deep examples include unquotation, an abstraction test, size, a normal-form test and CPS. They prove semantic correctness for the first four; the paper explicitly leaves the CPS correctness proof out, while presenting and testing the type-checked translation. The displayed CPS is call-by-name with an answer-polymorphic carrier; our local development fixes R. Their call-by-value variant is discussed but not displayed. Our local handlers and rank theorems are therefore not a substitute for a full PHOAS transformation theorem. The binder-free call-by-value fold has its own checked computation theorem and same-code constructor test, but covers a smaller domain.

The fidelity conventions also differ. Their operational presentation does not reduce type annotations, and their fidelity statement allows beta-equivalent type annotations. Here the original annotated PTS reduction does reduce them. The compatibility envelope preserves literal annotated syntax; the tag-free full dependent interface currently proves the weaker common-algebra conversion-reflection result. The simply typed and binder-free codes have intrinsic fidelity without that envelope. Derivation independence of the full quotation is another question, not a consequence of strong recovery.

Accordingly, the result is a *Brown–Palsberg-style indexed strong unquoter*, with a shared algebra API and genuine alternative folds. It is not yet a full dependent counterpart of their deep-embedding suite or their same-interface self-application experiments. Cross-index self-interpretation of a typed interpreter does not by itself give same-index self-application.

The next substantive goals are now sharply separated: construct a semantic-indexed dependent alternative algebra and its substitution laws; decode the actual dependent payload intrinsically; replace plan-specific coherence by a rank-only generic signature if possible; and derive reification from an independent accepted-proof predicate. For the combinatory route, original-source completeness and a dependent interpretation of dictionary atoms remain additional goals. None is settled by the fixed-head or conversion obstructions above.

All formal results use the original kernel and typing judgment. The development README and obligation ledger give the theorem interfaces; the latest handoff and verification records distinguish fresh scoped checks from earlier full-repository CI.